

CHEAT SHEET - Interface Pessoa-Máquina (IPM)

1. MODELO DE NORMAN

Fosso da Execução = Dificuldade em saber COMO agir | Distância entre intenções e ações disponíveis → **Minimizado por:** affordances claras, linguagem familiar, controles visíveis (NÃO por feedback!)
Fosso da Avaliação = Dificuldade em interpretar RESULTADO | Capacidade de determinar estado do sistema → **Minimizado por:** feedback imediato, indicadores de estado, confirmações **Princípios:** **Affordances** (indicam como usar) | **Mappings** (controles↔efeitos) | **Constraints** (restrições, podem ser +) | **Feedback** (resultado)

2. PRINCÍPIOS DE USABILIDADE

LEARNABILITY • Predictability: efeito futuro • Synthesizability: efeito passado • Familiarity: conhecimento prévio • Generalizability: extensão • Consistency: comportamentos	FLEXIBILITY • Dialogue initiative • Task migratability • Substitutivity • Customizability • Multithreading	ROBUSTNESS • Observability: estado interno • Recoverability: undo/redo • Responsiveness • Task conformance
--	---	--

NOVATOS: Familiarity, Predictability, Recoverability | **EXPERTS:** Synthesizability, Customizability, Flexibility

3. HEURÍSTICAS DE NIELSEN

1. Visibility of system status (feedback) | **2. Match system↔real world** (linguagem familiar) | **3. User control & freedom** (undo) **4. Consistency & standards** | **5. Error prevention** | **6. Recognition > recall** (minimizar memória) **7. Flexibility & efficiency** (atalhos experts) | **8. Aesthetic minimalist** | **9. Help recover errors** (msgs claras) | **10. Help & documentation**

4. MÉTODOS DE AVALIAÇÃO

ANALÍTICOS (sem utilizadores): • **Avaliação Heurística:** Peritos avaliam conformidade (3-5 avaliadores → 75% problemas) • **Cognitive Walkthrough:** Simular NOVATO por tarefas. Perguntas: Ação evidente? Controlo visível? Associação clara? **EMPIRÍCOS (com utilizadores):** • **Think-aloud:** Verbalizar pensamentos → identificar problemas específicos • **SUS:** 10 perguntas Likert 1-5, score 0-100, **>68 = acima média** • **NASA TLX:** Mede carga de trabalho mental → **comparar ESFORÇO** entre interfaces

5. WCAG + ACESSIBILIDADE

1. Perceivable: Alt-text, captions, contraste (4.5:1 AA) | **2. Operable:** Toda funcionalidade via TECLADO **3. Understandable:** Linguagem clara, comportamento previsível | **4. Robust:** HTML semântico, ARIA **WAI-ARIA:** role, aria-label, aria-expanded, aria-controls | **Regra #1:** HTML nativo ANTES de ARIA! **Screen readers:** JAWS, NVDA, VoiceOver | **<nav>** já é semântico (role="navigation" redundante)

6. CSS - FLEXBOX E GRID

FLEXBOX display: flex; flex-direction: row column; (row=DEFAULT) justify-content: center space-between; align-items: center flex-start flex-end; flex: 1 1 33%; /* grow shrink basis */ Media Queries: @media (min-width: 768px) {...} Mobile First: começar pelo mais restritivo	GRID display: grid; grid-template-columns: repeat(3, 1fr); repeat(auto-fit, minmax(250px, 1fr)); gap: 20px; /* auto-fit = responsivo SEM media queries */
--	---

7. GESTALT + DARK PATTERNS

Gestalt: **Proximidade** (próximos=relacionados) | **Similaridade** (semelhantes=agrupados) | **Fechamento** (completar formas) | **Continuidade** (caminhos suaves) **Dark Patterns (antiéticos!):** **Confirmshaming** (culpabilização) | **Roach Motel** (difícil sair) | **Misdirection** (desviar atenção) | **Forced action**

8. PROTÓTIPOS + HCD

Baixa fidelidade: Rápidos, baratos, testar conceitos, foco no conteúdo (não estética) | **Alta fidelidade:** Mais detalhados **Throw-away:** Exploração rápida | **Evolucionários:** Podem virar produto final | **Wireframes = baixa fidelidade HCD (ISO 9241-210):** Baseado em compreensão de utilizadores/tarefas/ambientes | Utilizadores envolvidos ao longo de TODO o processo Processo iterativo | Equipa multidisciplinar (não só UX) | Avaliação centrada no utilizador

CHEAT SHEET - Vue.js + Web Components

9. VUE.JS - CONCEITOS BASE

import { ref, reactive, computed, watch, onMounted, onUnmounted } from 'vue'; const count = ref(0); // primitivos → usa .value no script (NÃO no template) const state = reactive({x: 1}); // objetos (sem .value) - reactive(0) NÃO funciona! const doubled = computed(() => count.value * 2); // propriedade calculada (mais eficiente que recalculr) **Template:** v-model (two-way = v-bind:value + v-on:input) | v-for="item in items" :key="item.id" (-key OBRIGATÓRIO, usar id não index) v-if/v-else-if/v-else | @click="handler" (= v-on:click) | :prop="value" (= v-bind:prop)

10. VUE.JS - COMPONENTS

Props (pai→filho): defineProps(['message']) ou props: ['message'] | NÃO modificar props diretamente! **Emits (filho→pai):** defineEmits(['update']) → emit('update', payload) **v-model componentes:** :value="text" @input="\$emit('update:text', \$event.target.value)" **Lifecycle:** onMounted(() => { fetch inicial }) | onUnmounted(() => { clearInterval(id) }) - ESSENCIAL para cleanup! **Watch:** watch(count, (newVal, oldVal) => {}) - recebe novo E antigo | NÃO executa imediatamente (precisa immediate:true) **watchEffect:** tracking automático de dependências

11. PINIA - STATE MANAGEMENT

import { defineStore } from 'pinia'; export const useStore = defineStore('storeId', { state: () => ({ items: [] }), // SEMPRE função que retorna objeto getters: { count: (state) => state.items.length }, // propriedades calculadas actions: { addItem(item) { this.items.push(item); } // modificam estado (usam this) }); **■ ERROS COMUNS:** data: function(){} é Options API, NÃO Pinia! | getters não modificam estado | Em actions: this.items (não items = this)

12. VUE ROUTER

const routes = [{ path: '/', component: Home }, { path: '/user/:id', component: User, props: true }, // :id = parâmetro dinâmico { path: '/admin', component: Admin, children: [// nested routes { path: 'dashboard', component: Dashboard } // → /admin/dashboard]]]; **<router-link to="/">** → renderiza <a>, SEM recarregar página | **<router-link :to="{ name: 'user', params: { id: 1 } }">** **<router-view />** → onde componente é renderizado | **props: true** → passa params como props

13. FETCH API + PROMISES

// fetch retorna Promise com Response, .json() TAMBÉM retorna Promise fetch('http://api.com/items') .then(response => response.json()) // .json() é Promise! .then(data => this.list = data); // guardar em propriedade reativa // async/await (syntactic sugar sobre Promises) async function fetchData() { const response = await fetch('/api/users'); if (!response.ok) throw new Error(); // .ok = status 200-299 return await response.json(); } **■ ERRO:** this.list = fetch(...) guarda Promise, não dados! Usar .then() ou await

14. WEB COMPONENTS

Custom Elements: Nome DEVE ter hífen (my-element) | classe extends HTMLElement | customElements.define("my-el", Class) **Shadow DOM:** Encapsula estilos CSS | attachShadow({ mode: 'open' }) | <slot> injeta Light DOM no Shadow **Callbacks:** connectedCallback() = adicionado ao DOM | disconnectedCallback() = removido **NÃO requer framework** (Vue/React) - são nativos do browser!

15. REFERÊNCIA RÁPIDA - ERROS COMUNS

✗ ERROS FREQUENTES NOS TESTES: • data: function(){} é Options API de componentes, NÃO Pinia (Pinia usa state: () => {})) • getters em Pinia são para computed, NÃO para modificar estado (usar actions) • reactive(0) não funciona - reactive é só para objetos, usar ref(0) • Arrow function em addEventListener: this NÃO é o elemento clicado (usar function normal) • index como :key é problemático para listas que mudam - usar ID único • Observability pertence a ROBUSTNESS (não Flexibility) • Feedback minimiza fosso da AVALIAÇÃO (não execução) • watch NÃO executa imediatamente por defeito (precisa immediate: true) ✓ **LEMBRAR:** • ref() usa .value no script, mas NÃO no template | computed = mais eficiente • flex-direction: row é DEFAULT (não precisa declarar para horizontal) • <nav> já é semântico - role="navigation" é REDUNDANTE • Todas as opções de v-for funcionam: (item, index) in items | item in items | objetos também • gap em grid aplica-se a linhas E colunas | auto-fit = responsivo sem media queries • SUS > 68 = acima da média | NASA TLX = comparar esforço | Think-aloud = identificar problemas específicos